1. `fillna()` - Fill Missing Values

When data contains **NaN (Not a Number)** or missing values, `fillna()` is used to replace them.

Syntax

```
df.fillna(value)
```

Example Data

```
import pandas as pd
import numpy as np

df = pd.DataFrame({
    "Name": ["Krish", "Raj", "Amit"],
    "Marks": [90, np.nan, 85]
})

print(df)
```

Output:

```
    Name  Marks
0  Krish   90.0
1    Raj    NaN
2   Amit   85.0
```

Fill with 0

```
df.fillna(0)
```

Output:

```
    Name  Marks
0  Krish   90.0
1    Raj    0.0
2   Amit   85.0
```

Fill with Mean

```
mean_marks = df["Marks"].mean()

df["Marks"] = df["Marks"].fillna(mean_marks)
```

Calculation:

```
(90 + 85) / 2 = 87.5
```

Output:

```
    Name  Marks
0  Krish   90.0
1    Raj   87.5
2   Amit   85.0
```

Fill Different Columns

```
df.fillna({
    "Marks": 0,
    "Name": "Unknown"
})
```

Forward Fill (ffill)

Copies previous value.

```
df.fillna(method="ffill")
```

Example:

```
10
NaN
30
```

Output:

```
10
10
30
```

Backward Fill (bfill)

Copies next value.

```
df.fillna(method="bfill")
```

Example:

```
10
NaN
30
```

Output:

```
10
30
30
```

---

2. `dropna()` - Remove Missing Values

Removes rows or columns containing NaN values.

Example

```
import pandas as pd
import numpy as np

df = pd.DataFrame({
    "Name": ["Krish", "Raj", None],
    "Marks": [90, np.nan, 85]
})

print(df)
```

Output:

```
    Name  Marks
0  Krish   90.0
1    Raj    NaN
2   None   85.0
```

Remove Rows with Any NaN

```
df.dropna()
```

Output:

```
    Name  Marks
0  Krish   90.0
```

Remove Columns with NaN

```
df.dropna(axis=1)
```

`axis=0` → rows (default)

`axis=1` → columns

Remove Only If All Values Are NaN

```
df.dropna(how="all")
```

Remove If Any Value Is NaN

```
df.dropna(how="any")
```

Default behavior.

Specific Column Check

```
df.dropna(subset=["Marks"])
```

Removes rows where Marks is NaN.

Difference: fillna() vs dropna()

fillna()	dropna()
Replaces missing values	Removes missing values
Keeps data	Deletes data
Used when data is important	Used when row is useless

---

3. `groupby()` - Group Data

Most important function in Data Analysis.

Used to divide data into groups and perform calculations.

Example Dataset

```
import pandas as pd

df = pd.DataFrame({
    "Department": ["IT", "HR", "IT", "HR", "Sales"],
    "Salary": [50000, 40000, 60000, 45000, 55000]
})

print(df)
```

Output:

```
Department Salary
IT         50000
HR         40000
IT         60000
HR         45000
Sales      55000
```

Average Salary Department Wise

```
df.groupby("Department")["Salary"].mean()
```

Output:

```
Department
HR       42500
IT       55000
Sales    55000
```

Sum

```
df.groupby("Department")["Salary"].sum()
```

Output:

```
HR        85000
IT       110000
Sales     55000
```

Count

```
df.groupby("Department")["Salary"].count()
```

Output:

```
HR       2
IT       2
Sales    1
```

Multiple Aggregations

```
df.groupby("Department")["Salary"].agg(
    ["sum", "mean", "max", "min"]
)
```

Output:

```
             sum   mean   max   min
Department
```

```
HR         85000 42500 45000 40000
IT        110000 55000 60000 50000
Sales      55000 55000 55000 55000
```

Group By Multiple Columns

```
df.groupby(["Department", "Gender"])["Salary"].mean()
```

---

4. `merge()` - Join DataFrames

Works like SQL JOIN.

Student Data

```
students = pd.DataFrame({
    "ID": [1, 2, 3],
    "Name": ["Krish", "Raj", "Amit"]
})
```

Marks Data

```
marks = pd.DataFrame({
    "ID": [1, 2, 3],
    "Marks": [90, 85, 95]
})
```

Merge

```
pd.merge(students, marks, on="ID")
```

Output:

```
ID  Name   Marks
1   Krish   90
2   Raj     85
3   Amit    95
```

Inner Join

Only matching records.

```
pd.merge(students, marks,
         on="ID",
```

```
          how="inner")
```

Left Join

All left records.

```
pd.merge(students, marks,
          on="ID",
          how="left")
```

Right Join

All right records.

```
pd.merge(students, marks,
          on="ID",
          how="right")
```

Outer Join

All records from both.

```
pd.merge(students, marks,
          on="ID",
          how="outer")
```

Real Life Example

```
customers
orders
```

Join them:

```
pd.merge(customers,
          orders,
          on="customer_id")
```

---

5. `apply()` - Apply Function to Data

Used to execute a function on rows, columns, or values.

Example

```
df = pd.DataFrame({
    "Marks": [80, 90, 70]
})
```

Add 5 Marks

```
df["Marks"] = df["Marks"].apply(
    lambda x: x + 5
)
```

Output:

```
85
95
75
```

Convert to Grade

```
def grade(mark):
    if mark >= 90:
        return "A"
    elif mark >= 80:
        return "B"
    else:
        return "C"

df["Grade"] = df["Marks"].apply(grade)
```

Output:

```
Marks Grade
80      B
90      A
70      C
```

Apply on Entire Row

```
df["Total"] = df.apply(
    lambda row: row["Math"] + row["Science"],
    axis=1
)
```

axis=1

Row-wise operation.

---

axis=0

Column-wise operation.

---

apply() vs map()

apply()

Works on Series and DataFrame.

```
df["Marks"].apply(lambda x: x+5)
```

---

map()

Works only on Series.

```
df["Marks"].map(lambda x: x+5)
```

---

6. `to_csv()` - Save DataFrame to CSV File

Exports DataFrame into CSV.

Example

```
import pandas as pd

df = pd.DataFrame({
    "Name": ["Krish", "Raj"],
    "Marks": [90, 85]
})
```

Save CSV

```
df.to_csv("students.csv")
```

Creates:

```
students.csv
```

Content:

```
,Name,Marks
0,Krish,90
```

```
1,Raj,85
```

Remove Index

```
df.to_csv(
    "students.csv",
    index=False
)
```

Output:

```
Name,Marks
Krish,90
Raj,85
```

Custom Separator

```
df.to_csv(
    "students.csv",
    sep=";"
)
```

Output:

```
Name;Marks
Krish;90
Raj;85
```

Save Selected Columns

```
df.to_csv(
    "students.csv",
    columns=["Name"]
)
```

Output:

```
Name
Krish
Raj
```

Interview Summary

Function	Purpose
`fillna()`	Replace missing values
`dropna()`	Remove missing values

Function	Purpose
`groupby()`	Group data and perform calculations
`merge()`	Join multiple DataFrames
`apply()`	Apply custom function
`to_csv()`	Export DataFrame to CSV

Real Data Analysis Example

```
import pandas as pd

df = pd.read_csv("students.csv")

# Fill missing marks
df["Marks"] = df["Marks"].fillna(df["Marks"].mean())

# Remove rows where name is missing
df = df.dropna(subset=["Name"])

# Department-wise average marks
result = df.groupby("Department")["Marks"].mean()

# Add Grade column
df["Grade"] = df["Marks"].apply(
    lambda x: "Pass" if x >= 35 else "Fail"
)

# Save final data
df.to_csv("final_report.csv", index=False)

print(result)
```

This example combines **fillna() + dropna() + groupby() + apply() + to_csv()**, which is a very common workflow in Data Analysis projects.

Link : CHATGPT

Link : ALL